

Sistema de verificación de hechos

Jose Rodriguez, Miguel Carballo, Mario Rodriguez y David Gonzalez

7 de enero de 2026

1. Introduccion

La aplicación implementada es un sistema de verificación de hechos que, utilizando un sistema RAG, es capaz de verificar la veracidad de declaraciones de acuerdo con un corpus específico.

El sistema comprueba si la declaración es verdadera, falsa, o si no hay suficiente información en el corpus para evaluarla. La respuesta debe ir acompañada de los fragmentos del corpus que justifiquen el veredicto dado. Adicionalmente, se han implementado una serie de funcionalidades adicionales, que permiten la generación de la confianza que tiene en su evaluación de la declaración, la generación de un resumen del contexto que ha utilizado para la evaluación, y el manejo de varios idiomas, traduciendo la declaración al inglés, para luego procesarla y traducirla de vuelta al idioma de la declaración o a otro idioma especificado en la aplicación.

El repositorio de GitHub en el que se ha desarrollado la práctica es el siguiente: [Repositorio de GitHub](#).

2. Decisiones de Diseño

Durante el desarrollo del sistema, se han tomado una serie de decisiones acerca de las tecnologías a usar para el funcionamiento. El LLM al que se realizan las consultas es el entorno de despliegue de la UC3M, que tiene un modelo llama. Para la interacción con el LLM usado, se ha utilizado la librería ollama. El corpus para la verificación de consultas está compuesto de más de 100 artículos de Wikipedia de diversos tópicos, como la informática, biología, historia, etc. Para el procesamiento de estos artículos se utiliza la API de Wikipedia y se guardan en ChromaDB, una base de datos vectorizada, de forma que la extracción de datos sea más eficiente gracias a guardar los datos como embeddings.

Para minimizar las alucinaciones, se utiliza un sistema RAG, de forma que el modelo obtenga la información necesaria para la verificación de consultas solamente de la base de datos. Este sistema recoge los fragmentos de texto más relevantes de

acuerdo con la consulta del usuario para que el LLM pueda generar una respuesta basándose solamente en la información recuperada.

3. Implementación de los requisitos funcionales

Para la implementación de los requisitos funcionales, en primer lugar se genera la base de datos utilizando el fichero `ingest.py`, que realiza los embeddings de los artículos que vienen especificados en la lista de títulos de artículos de Wikipedia del fichero `config.py`. Después, se ejecuta la aplicación, cuyo front end se encuentra en el fichero `.app.py`, el cual se comunica con el backend, el fichero `rag_engine.py`. En este último fichero se especifica el modelo del entorno de despliegue de la UC3M y se realizan una serie de prompts para el correcto funcionamiento de la aplicación.

3.1. Prompt

Explicación del prompt y de su proceso de desarrollo hasta ser funcional (meter ejemplos, instrucción de ignorar información externa, instrucción de sistema y de usuario, temperatura).

Se utilizan distintos prompts para las funcionalidades implementadas. Los requisitos funcionales cuentan con dos prompts, un prompt de verificación de hechos, que comprueba si la declaración es verdadera, falsa, o no hay suficiente información, y un prompt que extrae el fragmento de texto que se ha usado para el veredicto en caso de que la declaración sea verdadera o falsa. Cada uno de estos prompts está dividido en dos, un primer prompt que se introduce en el LLM con el rol de sistema, el cual indica las instrucciones y criterios que debe seguir el modelo, y un segundo prompt con el rol de usuario, que tiene la declaración y el contexto extraído por el RAG. Los prompts tienen instrucciones que especifican cuándo se debe dar cada veredicto, como por ejemplo, que el veredicto sea falso cuando la declaración contradice el contexto. Además, tienen una instrucción importante que indica que solamente se debe tener en cuenta la información del contexto, ignorando cualquier

conocimiento del mundo real, incluso verdades o falsedades universales, siempre que no vengan especificadas en el contexto, ya que, de no especificar esto, el modelo fallaba con este tipo de declaraciones. También se ha optado por hacer *Few-Shot Prompting*, añadiendo una serie de ejemplos de respuestas, de forma que el modelo tenga claro tanto el formato de repuesta como qué veredictos dar en ciertos escenarios. Por último, se ha establecido una temperatura muy baja en el LLM, siendo esta 0 en algunos prompts, de forma que el veredicto del modelo sea determinista.

4. Funcionalidades adicionales

Se han implementado tres funcionales adicionales en el sistema. En primer lugar, se ha añadido la confianza del sistema en el veredicto, en forma de un porcentaje. Este se genera mediante un prompt, que establece una serie de rangos para el porcentaje de acuerdo a cómo de seguro está de la afirmación. El prompt funciona de la misma manera que los prompts del apartado anterior, teniendo uno de sistema y otro de usuario, y contando tanto con instrucciones claras como con ejemplos. Por otra parte, se ha implementado la generación de resúmenes, que se generan utilizando el propio LLM, también mediante un prompt de sistema y otro de usuario, en el que se pide al LLM que genere un resumen del contexto extraído por el sistema RAG, limitando el número de palabras del resumen a estar entre 40 y 100. Por último, se ha implementado la funcionalidad de traducción, que permite manejar consultas y respuestas en varios idiomas usando modelos de traducción preentrenados. Se detecta el idioma en el que se ha escrito la declaración, esta se traduce al inglés para ser introducida en el LLM, y la respuesta en inglés del LLM es traducida del vuelta al idioma detectado. Para la detección del idioma de la declaración, se ha utilizado el modelo *papluca/xlm-roberta-base-language-detection*, y para la traducción se ha utilizado el modelo *facebook/nllb-200-distilled-600M*. El primero utiliza el formato de idiomas ISO, mientras que el segundo utiliza el formato NLLB, por lo que es necesario traducir el formato. Se establece una longitud máxima de 512 para que no haya problemas de truncación, y se traducen las respuestas a cada prompt por separado, tanto el veredicto, como el fragmento extraído del contexto como el resumen de este. También se ha añadido un campo en la aplicación que permite establecer un idioma específico para la respuesta del LLM independientemente del idioma de la de-

claración.

5. Evaluación del sistema

La evaluación de desempeño se realizó mediante un dataset de 100 afirmaciones en diferentes idiomas para validar el RAG. El conjunto de prueba incluyó un 30 % de afirmaciones verdaderas (extraídas de Wikipedia), un 30 % de negaciones lógicas (falsas) y un 40 % de afirmaciones aleatorias no presentes en los temas seleccionados (insuficiente).

Los resultados comparativos entre los modelos evaluados muestran una **precisión de 1.00** en las categorías *True* y *False* para ambos LLM probados (Qwen3 y llama3), garantizando la fiabilidad de los veredictos. El sistema basado en **Llama 3** alcanzó un *accuracy* global del **61 %**, especialmente la confirmación de verdades con (*Recall* de 0.57). **Qwen 2.5**, aunque obtuvo un *accuracy* ligeramente inferior (58 %), tuvo mayor sensibilidad ante contradicciones, aumentando la tasa de detección de falsedades (*Recall* de la categoría *False*) de un 0.13 a un **0.20**. El sistema presenta un sesgo conservador, si no obtiene evidencia absoluta de la afirmación se ve mas orientado a categorizarlo como *Insufficient Information*.

6. Conclusiones y limitaciones

El proyecto valida que la arquitectura RAG es capaz de actuar como un verificador de hechos multilingüe fiable utilizando modelos locales. Se concluye que la fidelidad a la fuente no depende únicamente del LLM, sino de la configuración del motor de búsqueda vectorial, ya que ambos modelos mantuvieron un buen rendimiento. A continuación se muestran las limitaciones del sistema:

- **Recall en desmentidos:** El sistema presenta dificultades para marcar una afirmación como *False* si la base de datos no contiene una contradicción semántica explícita, tendiendo a la neutralidad.
- **Dependencia Lingüística:** El pipeline de traducción está limitado por el diccionario de mapeo de códigos ISO a **NLLB-200**; idiomas no preconfigurados impiden la ejecución del proceso.
- **Ventana de Contexto:** La restricción de búsqueda a $k = 3$ documentos limita la capacidad de síntesis en afirmaciones que requieren información distribuida en múltiples artículos de Wikipedia.